

Lab Assignment No.	05
Title	Optimization of genetic algorithm parameter in hybrid genetic algorithm-neural network modelling: Application to spray drying of coconut milk.
Roll No.	
Class	BE AI & DS
Date Of Completion	
Subject	Computer Laboratory III[417534]
Assessment Marks	
Assessor's Sign	

Assignment No. 05

Title: Optimization of Genetic Algorithm Parameters in Hybrid Genetic Algorithm-Neural Network Modelling: Application to Spray Drying of Coconut Milk

Problem Statement: Optimize the parameters of a genetic algorithm in a hybrid genetic algorithm-neural network (GA-NN) model to enhance the accuracy and efficiency of spray drying process optimization for coconut milk.

Prerequisites: Basic knowledge of genetic algorithms, neural networks, hybrid optimization techniques, machine learning, and Python programming. Familiarity with Jupyter Notebook and scientific computation libraries such as NumPy, TensorFlow, and SciPy.

Software Requirements: Windows/Linux/macOS, Python (NumPy, TensorFlow/Keras, SciPy, Matplotlib), Jupyter Notebook, and Scikit-learn for model evaluation.

Hardware Requirements: Intel Core i5 or higher, 8GB RAM, 500MB free disk space, and an active Python environment.

Theory:

Introduction to Genetic Algorithm (GA) and Neural Networks (NN):

Genetic Algorithm (GA) is an optimization technique inspired by natural selection, commonly used to find optimal solutions in search spaces. Neural Networks (NN) are machine learning models designed to recognize patterns and relationships in data. When combined, GA optimizes the hyperparameters of NN models to improve their performance.

Hybrid Genetic Algorithm-Neural Network Modelling:

1. Genetic Algorithm (GA) in Optimization:

- GA works by evolving a population of candidate solutions using selection, crossover, and mutation operations.
- The objective function evaluates model performance, and the best individuals are selected for the next generation.

2. Neural Network (NN) for Spray Drying Modelling:

- NN models predict outcomes in spray drying processes based on input parameters such as temperature, air velocity, and feed rate.
- Optimizing NN hyperparameters (e.g., learning rate, number of layers, number of neurons) enhances predictive accuracy.

Optimization Process in GA-NN:

1. **Encoding:** Represent NN hyperparameters as a chromosome.
2. **Fitness Evaluation:** Train and validate the NN using different parameter sets, evaluating performance based on error metrics.
3. **Selection:** Choose the best-performing individuals.
4. **Crossover & Mutation:** Generate new candidates by combining and slightly modifying existing parameters.
5. **Iteration:** Repeat the process until the optimal parameter set is found.

Application to Spray Drying of Coconut Milk:

- Spray drying is a process used to convert liquid coconut milk into powder form.
- The optimization of process parameters (e.g., inlet air temperature, feed concentration) is crucial for maximizing yield and quality.
- The GA-NN model improves predictive capability, enabling better control over drying conditions.

Implementation Approach:

1. **Data Collection:** Gather experimental spray drying data.
2. **Preprocessing:** Normalize and split data for training/testing.
3. **GA Implementation:** Initialize and evolve a population of NN hyperparameter sets.
4. **Train & Evaluate NN:** Use optimized parameters to train the NN.
5. **Performance Analysis:** Compare GA-optimized NN results with standard models.
6. **Visualization:** Use Matplotlib to plot optimization trends and results.

Applications of GA-NN Hybrid Models:

- Optimization of industrial processes.
- Predictive modeling in food processing.

- AI-driven quality control in manufacturing.
- Adaptive control systems for process automation.

Source Code –

```
# Import Required Libraries
```

```
import numpy as np import pandas as pd
```

```
import tensorflow as tf from tensorflow.keras.models
```

```
import Sequential from tensorflow.keras.layers
```

```
import Dense from sklearn.model_selection
```

```
import train_test_split from sklearn.metrics
```

```
import mean_squared_error from deap
```

```
import base, creator, tools, algorithms
```

```
import random import matplotlib.pyplot as plt
```

```
import warnings warnings.filterwarnings("ignore", category=UserWarning)
```

```
# Load and Preprocess Data
```

```
# Simulate dataset (replace with real-world data if available)
```

```
np.random.seed(42)
```

```
n_samples = 100
```

```
X = np.random.uniform(low=30, high=100, size=(n_samples, 2)) # e.g., temperature, feed rate
```

```
y = 0.8 * X[:, 0] - 0.5 * X[:, 1] + np.random.normal(0, 5, n_samples) # Spray drying efficiency
```

```
# Split the dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Normalize inputs
```

```
X_mean, X_std = X_train.mean(axis=0), X_train.std(axis=0)
```

```
X_train = (X_train - X_mean) / X_std
```

```
X_test = (X_test - X_mean) / X_std
```

```
# Define Neural Network
```

```
def build_nn(num_neurons, learning_rate):
```

```
    model = Sequential([ Dense(num_neurons, activation='relu', input_shape=(X_train.shape[1],)),  
                        Dense(1) ])
```

```
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate), loss='mse')
```

```
    return model
```

```
# Genetic Algorithm for Optimization
```

```
# Define GA to optimize number of neurons and learning rate
```

```
def evaluate_fitness(individual):
```

```
    num_neurons = int(individual[0]) # Number of neurons
```

```
    learning_rate = individual[1] # Learning rate
```

```
    model = build_nn(num_neurons, learning_rate)
```

```
    model.fit(X_train, y_train, epochs=20, verbose=0, batch_size=10)
```

```
# Predict and calculate mean squared error on test set
```

```
y_pred = model.predict(X_test, verbose=0)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
    return mse,
```

```
# GA Configuration
```

```
toolbox = base.Toolbox()
```

```
creator.create("FitnessMin", base.Fitness, weights=(-1.0,)) # Minimize MSE
```

```
creator.create("Individual", list, fitness=creator.FitnessMin)
```

```
toolbox.register("attr_num_neurons", random.randint, 5, 50) # Range for neurons
```

```
toolbox.register("attr_learning_rate", random.uniform, 0.001, 0.01) # Range for learning rate
```

```
toolbox.register("individual", tools.initCycle, creator.Individual, (toolbox.attr_num_neurons,  
                                                                    toolbox.attr_learning_rate), n=1)
```

```
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```

```
toolbox.register("mate", tools.cxBlend, alpha=0.5)
```

```
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=0.1, indpb=0.2)

toolbox.register("select", tools.selTournament, tournsize=3) toolbox.register("evaluate",
evaluate_fitness)

# Run GA

random.seed(42)

population = toolbox.population(n=10)

ngen = 20

cxpb, mutpb = 0.5, 0.2

result, log = algorithms.eaSimple(population, toolbox, cxpb=cxpb, mutpb=mutpb, ngen=ngen,
verbose=True)

# Best Solution

best_individual=tools.selBest(population,k=1)[0]

print(f"BestIndividual: Neurons={best_individual[0]}, LearningRate={best_individual[1]}")

#Train and evaluate the final model

final_model =build_nn(int(best_individual[0]), best_individual[1])

final_model.fit(X_train,y_train, epochs=50, verbose=1, batch_size=10)

#Test the model

y_pred = final_model.predict(X_test)

final_mse = mean_squared_error(y_test, y_pred)

print(f"Final MSE on Test Data: {final_mse}")

# Visualize Results

# Plot true vs predicted values

plt.scatter(y_test, y_pred, c='blue', label='Predictions')

plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linestyle='--')

plt.xlabel("TrueValues")

plt.ylabel("Predicted Values")
```

```
plt.legend()
```

```
plt.title("True vs Predicted Values")
```

```
plt.show()
```

Conclusion: This practical demonstrates the application of a hybrid GA-NN model for optimizing genetic algorithm parameters in spray drying of coconut milk. The approach enhances predictive accuracy and improves process efficiency through AI-driven optimization techniques.